

VBASE to construct good plans by reusing PostgreSQL’s built-in mechanism.

Figure 8a shows execution times of different planning strategies for varying scalar selectivities and a fixed vector filter selectivity. The default strategy estimates selectivity as 0.5 as PASE does. The result demonstrates that VBASE can produce execution plans that closely match the ground truth of the best choice of index scan strategies. When scalar selectivity is less than 0.18, VBASE predicts accurately that the cost of execution via B-tree index is smaller than vector index traversal, and chooses to run it. Likewise, if scalar selectivity is over 0.18, VBASE correctly chooses vector index traversal.

On the other hand, experiments in Figure 8b vary vector filter selectivities and fix scalar filter selectivity. Like in Figure 8a, the result shows VBASE can create the high efficacy of query planning based on highly accurate cost estimation.

In comparison, systems like PASE do not provide an accurate estimation for selectivity. And they do not tune cost estimation for vector computation and vector index traversal either. The inaccurate estimation causes PASE to always execute queries via B-tree index in this experiment.

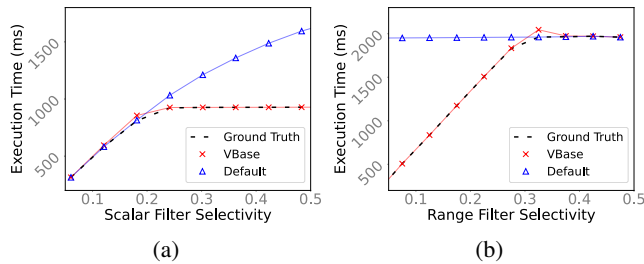


Figure 8: Query execution time with different estimation. We fixed range filter selectivity=0.13 in (a), and scalar filter selectivity=0.90 in (b)

6 Related Works

Similarity Query in Databases. Several works [20, 21, 71, 73, 74] have explored extending database systems to support accurate similarity query of low-dimensional vector data, in which K -NN($TopK$) and *range filter* query are well described. R-Tree [38], KD-Tree [34], M-Tree [27], Slim-Trees [47] can be used in these works as indices for low-dimensional data. [20] proposes to include similarity queries to SQL and run them on SIREN [21]. [71] presents similarity Join and similarity Group-by operators. The similarity Group-by operator for high-dimensional vector is actually equivalent to the clustering operation, which has been well-studied by [15, 18, 50, 65]. However, all of these works are about clustering data on the main table instead of the vector indices.

Vector Indices. Vector indices support approximate nearest neighbor search efficiently on high-dimensional vector by $TopK$ interface. They can be divided into two categories: graph-based approach and partition-based approach. The

partition-based approach divides the whole vector space into many sub-spaces, and uses some metric (e.g. a centroid, a hash value or a divisional plane) to represent all vectors that belong to a sub-space. During the traverse, it navigates a query to its approximate nearest sub-spaces step by step based on distances between the query and the representative metric of sub-spaces. Representative partition-based approaches include clustering-based solutions [5, 17, 19, 25, 44, 45, 48, 90], hash-based solutions [30, 41, 79, 81, 85], and tree-based solutions [23, 57, 62, 78]. The graph-based approach represents each vector as a vertex, each connected to its nearest vectors (i.e. neighbors) by edges in a graph. There are also some shortcut edges connecting to distant vector vertices, which can speed up the graph traversals. Using this neighborhood graph, index traversal can be guided by a query approximately towards its closest neighbors step by step from a fixed starting point [32, 39, 43, 55, 58, 77]. $TopK$ interface in vector indices has limited query expressiveness.

Vector Databases based on $TopK$. AnalyticDB-V [80], PASE [86], Milvus [76], and Elasticsearch [4] support complex vector queries based on the original $TopK$ interface in vector indices. AnalyticDB-V [80] and PASE [86] integrate vector indices into the database engine to support SQL interface for similarity queries. Elasticsearch [4] is a distributed full-text search engine, providing approximate nearest neighbor search based on HNSW [89]. AnalyticDB-V [80], PASE [86], and Elasticsearch [4] begin to support vector search plus scalar attribute filtering. Milvus [76] is a data management system to efficiently manage large-scale vector data, which can additionally support multi-column $TopK$ queries by iteratively speculating the K with a growing value. In contrast, VBASE does not rely on a tentative index collected by $TopK$.

7 Conclusion

This paper presents VBASE, a vector database that integrates high-dimensional vector indices into PostgreSQL, a relational database to facilitate complex approximate similarity queries. Unlike conventional approaches that leverage $TopK$ to collect the target vector’s K nearest neighbors where a conventional index is constructed for query execution, VBASE builds on relaxed monotonicity, a common foundation between conventional and high-dimensional indices. This common foundation allows VBASE to build a unified query execution engine that produces query results equivalent to those produced by $TopK$ -based solutions with the optimal $\hat{K}$. As a result, VBASE significantly outperforms state-of-the-art vector systems on complex vector queries.

8 Acknowledgement

We would like to thank our shepherd Marco Serafini and the anonymous reviewers for their insightful comments.